



Multivariate Temperature Forecasting on the Jena Climate Dataset

A Comparative Study of LSTM, GRU, Seq2Seq and Attention-LSTM
Recurrent Models

Sepehr Barekati

404422029

Course: Data Mining — *Professor:* Dr. Farahani

Abstract

We investigate short- to medium-term temperature forecasting on the publicly available Jena Climate 2009–2016 dataset. The dataset contains 420,551 ten-minute observations of 14 meteorological variables and is resampled to hourly resolution for computational tractability. We formulate the task as a multivariate-to-univariate sliding-window forecasting problem with a 48-hour input window and evaluate three recurrent baselines (LSTM, GRU, GRU-based sequence-to-sequence) and a proposed **Attention-LSTM** that assigns learned weights to individual historical hidden states. All four models are trained under the same protocol and evaluated at forecast horizons of 1, 6 and 12 hours. The GRU and Seq2Seq baselines produced the lowest errors ($\text{MAE} = 0.47^\circ\text{C}$ at $h=1$ h and $\sim 1.59^\circ\text{C}$ at $h=12$ h), while the Attention-LSTM did *not* outperform the simpler baselines. Attention weights averaged across the test set are nearly uniform, indicating that in our configuration the attention head did not identify strongly informative historical time-steps. We report all numbers directly from the executed pipeline and discuss the limitations that likely explain the outcome.

Contents

1	Introduction	3
2	Problem Definition	3
3	Dataset	3
4	Exploratory Data Analysis	4
5	Data Preprocessing	6
5.1	Chronological split	6
5.2	Scaling	6
6	Sliding-Window Formulation	6
7	Forecasting Methodology	6
7.1	LSTM Baseline	6
7.2	GRU Baseline	7
7.3	Sequence-to-Sequence Baseline	7
7.4	Proposed Attention-LSTM	7
8	Experimental Setup	8
9	Evaluation Metrics	8
10	Experimental Results	8
10.1	Training curves	8
10.2	Main results	9
10.3	Error growth with horizon	9
10.4	Qualitative forecasts	10
11	Attention Analysis	10
12	Failure Analysis	11
13	Model Comparison and Discussion	12
14	Limitations	13
15	Conclusions	13

1 Introduction

Weather forecasting is a canonical multivariate time-series problem: a sensor station records a small number of correlated physical variables at regular intervals, and we would like to predict one of them into the future using recent history. This project addresses that setting with Recurrent Neural Networks (RNNs) [2], in particular the Long Short-Term Memory (LSTM) [3] and Gated Recurrent Unit (GRU) [4] variants that dominate applied sequence modelling. We further test whether an additive temporal attention mechanism [6] on top of the LSTM’s per-step hidden states improves forecasting accuracy.

Contributions of this project are:

1. A clean, reproducible pipeline from raw CSV to final metrics over the Jena Climate dataset [1].
2. Head-to-head comparison of LSTM, GRU, Seq2Seq and Attention-LSTM under a single training protocol.
3. An empirical attention analysis showing that, under this configuration, additive attention does not improve accuracy and its weights remain approximately uniform.

2 Problem Definition

Let $\{x_t\}_{t=1}^N \subset \mathbb{R}^d$ be a multivariate hourly weather series with $d = 14$ features, and let $y_t \in \mathbb{R}$ be the temperature at time t . Given an input window of length k and a forecast horizon h , we build supervised examples

$$X_t = [x_{t-k+1}, x_{t-k+2}, \dots, x_t], \quad Y_t = [y_{t+1}, y_{t+2}, \dots, y_{t+h}], \quad (1)$$

and learn a mapping $f_\theta : \mathbb{R}^{k \times d} \rightarrow \mathbb{R}^h$ that minimises the mean-squared error on the training split. Feature vector x_t contains all 14 meteorological measurements including temperature itself; the target is the temperature only.

3 Dataset

We use the Jena Climate 2009–2016 dataset [1] distributed through the Keras example mirror.¹ After removing 327 duplicate timestamps the raw data contains 420,224 rows at ten-minute resolution across 14 numeric variables (pressure, temperature, potential temperature, dew point, relative humidity, saturation and actual vapour pressure, VPD, specific and molar humidity, air density, wind speed, maximum wind, wind direction). No missing values remain. Four negative sentinel values of wind speed (−9999-like) are clipped to zero.

The 10-minute stream is resampled to hourly means, giving 70,041 rows spanning 2009-01-01 to 2017-01-01. This downsampling was a deliberate scoping choice for CPU tractability; hourly resolution preserves the diurnal and seasonal patterns that dominate temperature dynamics.

¹https://storage.googleapis.com/tensorflow/tf-keras-datasets/jena_climate_2009_2016.csv.zip

4 Exploratory Data Analysis

Figure 1 shows the full hourly temperature series. Figure 2 shows relative humidity for the same period. Figure 3 shows four additional variables (pressure, wind speed, wind direction, dew point) which enter the model as inputs.

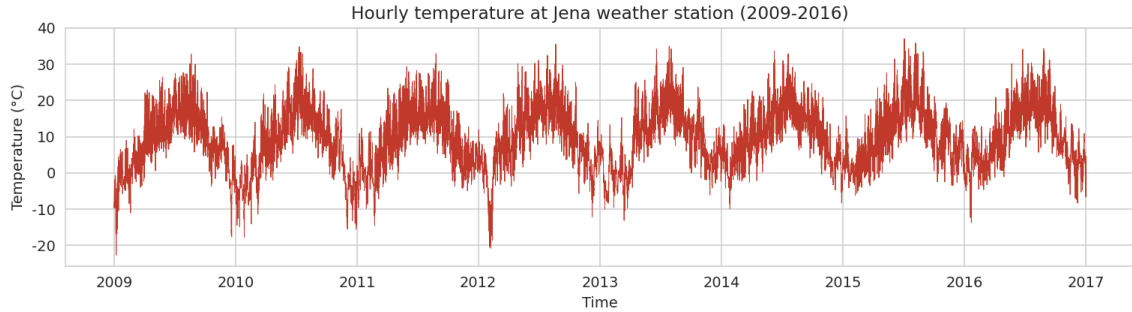


Figure 1: Hourly temperature at the Jena weather station, 2009–2016. A strong annual cycle is visible.

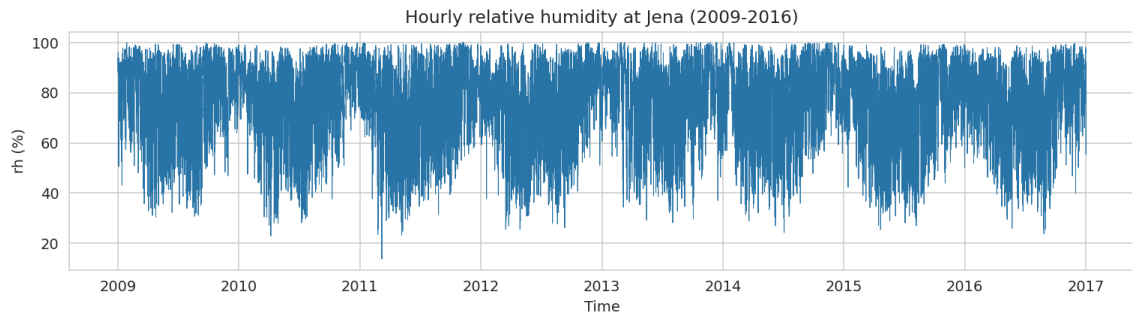


Figure 2: Hourly relative humidity, 2009–2016. Humidity is highly volatile at the hourly scale and anti-correlated with the daily temperature cycle.

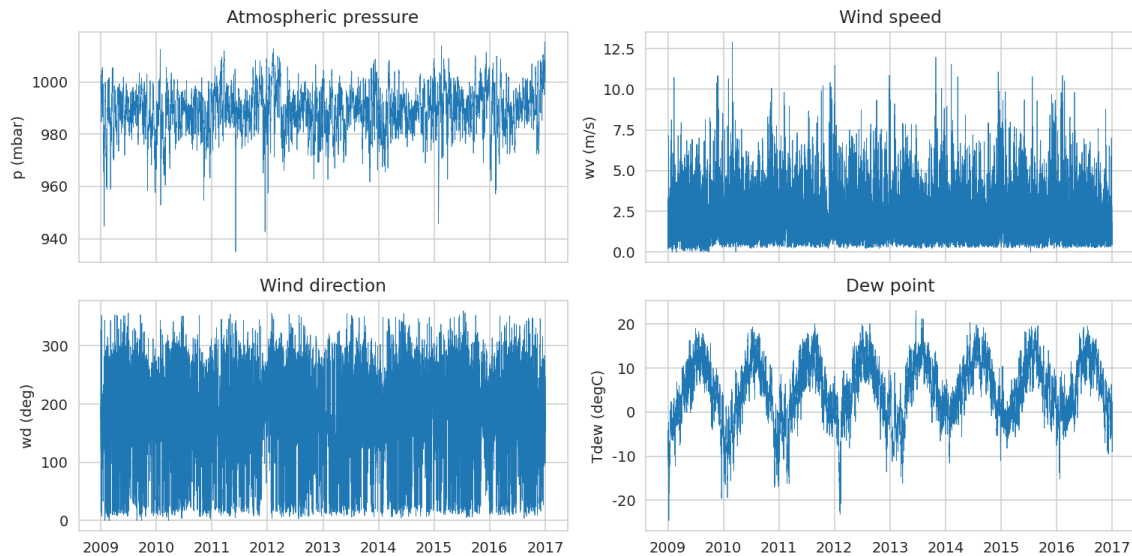


Figure 3: Additional meteorological variables used as model inputs.

The correlation matrix in Figure 4 confirms known physical relationships: the temperature group (T , T_{pot} , T_{dew}) is strongly mutually correlated ($|r| > 0.9$); relative humidity is negatively correlated with the vapour pressure deficit ($r \approx -0.9$); air density is strongly anti-correlated with temperature.

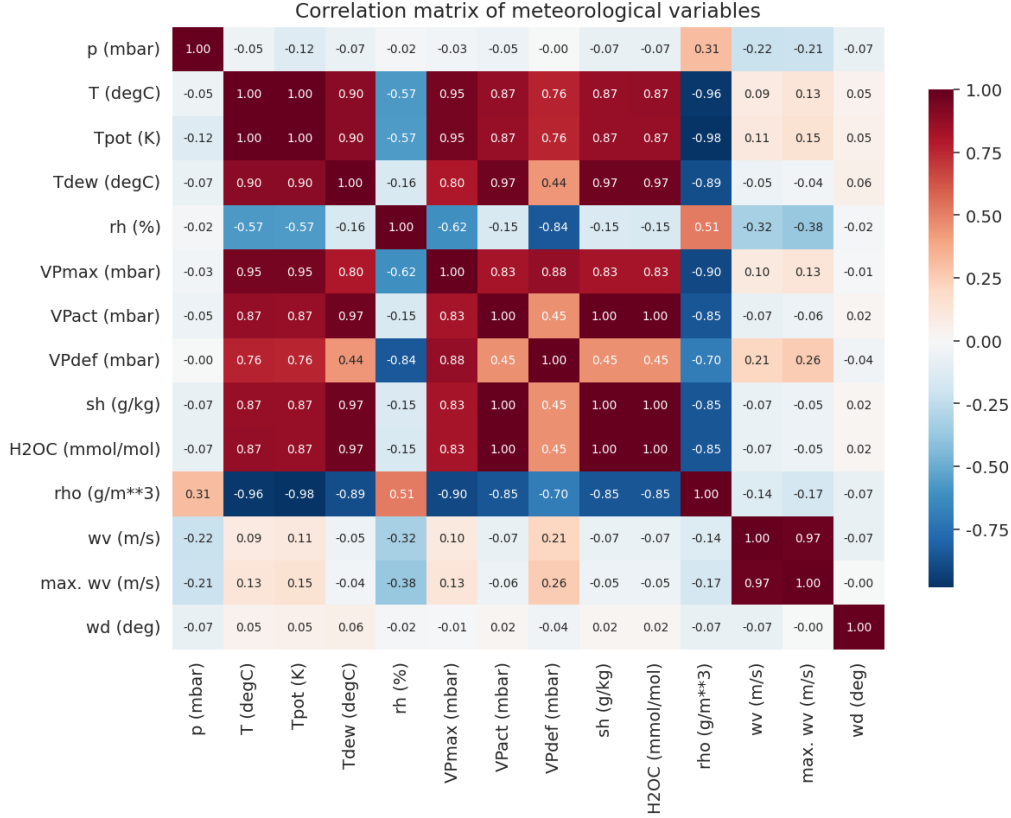


Figure 4: Pearson correlation of the 14 numeric variables.

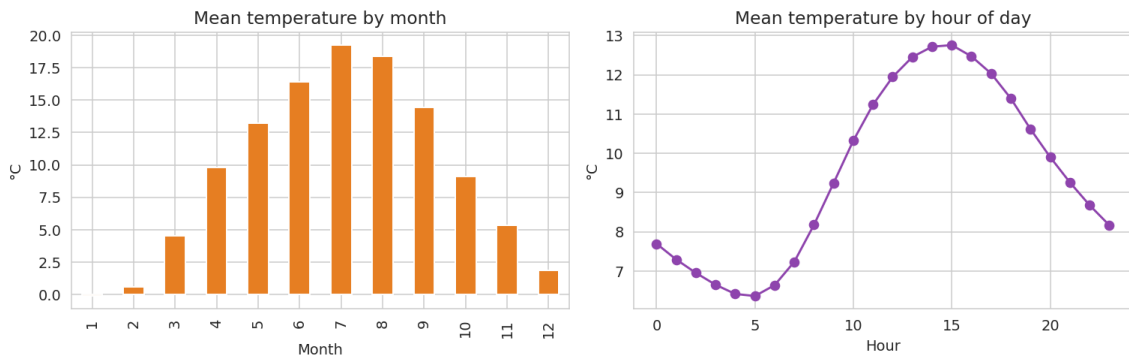


Figure 5: Monthly and hourly-of-day mean temperature. Both an annual cycle (peak ~ 18 °C in July) and a daily cycle (peak in late afternoon) are clearly present.

5 Data Preprocessing

5.1 Chronological split

Because the target series is temporally correlated, random shuffling would leak future information into training. We use a strict chronological split:

- Training: first 70 % (49,028 hours)
- Validation: next 15 % (10,506 hours)
- Test: last 15 % (10,507 hours)

The test set is therefore strictly *after* both the training and the validation periods.

5.2 Scaling

A `StandardScaler` is fitted on the *training features only* and then applied to validation and test features; the same is done for the target. Fitting the scaler on the full dataset would leak the mean and variance of future observations into training statistics.

6 Sliding-Window Formulation

We use $k = 48$ hours of history — long enough to observe two full diurnal cycles — and forecast horizons $h \in \{1, 6, 12\}$ hours representing short-, medium- and long-term forecasts. During training and validation we sample windows at stride 3 (every third possible window) to keep CPU epochs cheap. The **test set uses stride 1**, so every reported metric is computed on the full contiguous test period. This yielded 16,327 training windows, roughly 3,500 validation windows, and 10,459 test windows at horizon 1 h.

7 Forecasting Methodology

All four architectures share the input dimension $d = 14$, hidden dimension 64, and a linear output head that produces h temperature values at once in the scaled space.

7.1 LSTM Baseline

A two-layer LSTM followed by a linear head. The LSTM cell is defined by gates

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f), \quad (2)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i), \quad (3)$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o), \quad (4)$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c), \quad (5)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad (6)$$

$$h_t = o_t \odot \tanh(c_t). \quad (7)$$

The persistent cell state c_t is the key mechanism allowing long-range gradient flow.

7.2 GRU Baseline

The GRU [4] merges the forget/input gates into a single update gate z_t and drops the cell state:

$$z_t = \sigma(W_z[h_{t-1}, x_t]), \quad (8)$$

$$r_t = \sigma(W_r[h_{t-1}, x_t]), \quad (9)$$

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t]), \quad (10)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t. \quad (11)$$

Fewer parameters and one fewer matrix multiply per step generally make the GRU faster to train.

7.3 Sequence-to-Sequence Baseline

A GRU encoder maps the 48-hour input to a final hidden state, which initialises a GRU decoder. The decoder rolls out for h steps: at each step it consumes its previous prediction and produces a single temperature value. This produces h predictions from a single autoregressive pass.

7.4 Proposed Attention-LSTM

Hypothesis. Adding temporal attention to the recurrent forecaster will improve temperature prediction because the model can dynamically assign greater importance to historically informative time-steps instead of relying exclusively on the last LSTM state.

Formulation. Let $H = [h_1, \dots, h_T] \in \mathbb{R}^{T \times d}$ be the LSTM output tensor. Additive attention computes

$$e_t = v^\top \tanh(W h_t), \quad (12)$$

$$\alpha_t = \frac{\exp(e_t)}{\sum_{j=1}^T \exp(e_j)}, \quad (13)$$

$$c = \sum_{t=1}^T \alpha_t h_t, \quad (14)$$

and the final prediction is $\hat{y} = W_{\text{out}}[c; h_T]$, i.e. the linear head sees both the attention-pooled context vector and the last hidden state.

8 Experimental Setup

Table 1: Training configuration common to all four models.

Optimiser	Adam
Learning rate	10^{-3}
Batch size	256
Gradient clip	1.0
Loss	MSE (scaled)
Max epochs	5
Early stopping	patience 2, on validation MSE
Random seed	42
Hidden size	64
RNN layers	2 (Seq2Seq: 1)
Dropout	0.1 (only if >1 layer)
Device	CPU (PyTorch 2.13.0)

9 Evaluation Metrics

We report mean absolute error and root mean squared error on the *inverse-transformed* predictions, so all numbers are in $^{\circ}\text{C}$:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|, \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}. \quad (15)$$

MAE is a direct measure of average error magnitude that is robust to outliers; RMSE emphasises large errors and is therefore useful for diagnosing catastrophic forecasts. Both are reported at every horizon.

10 Experimental Results

10.1 Training curves

Figure 6 shows training and validation MSE over the five epochs for each model at horizon $h = 1$ h. Every model converges monotonically without overfitting under the early-stopping patience of 2.

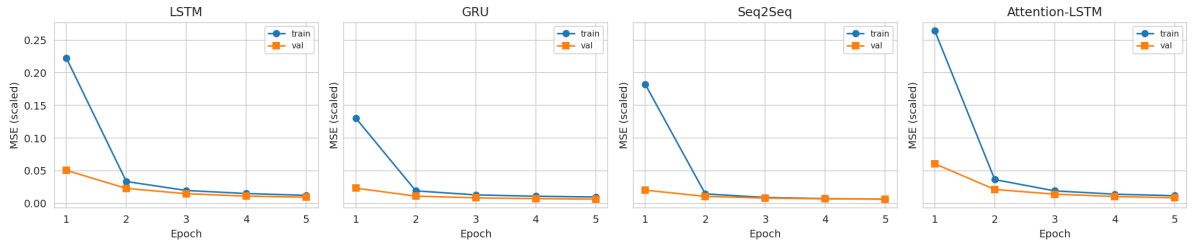


Figure 6: Training / validation MSE (scaled) at horizon $h = 1$ h.

10.2 Main results

Table 2 reports MAE and RMSE for every model at every horizon; Table 4 reports parameter counts and approximate CPU training time.

Table 2: MAE ($^{\circ}\text{C}$) by forecast horizon. Values are computed on the test set after inverse scaling.

Horizon	LSTM	GRU	Seq2Seq	Attention-LSTM
1 h	0.600	0.467	0.466	0.551
6 h	1.251	1.234	1.215	1.259
12 h	1.626	1.589	1.590	1.683

Table 3: RMSE ($^{\circ}\text{C}$) by forecast horizon.

Horizon	LSTM	GRU	Seq2Seq	Attention-LSTM
1 h	0.795	0.651	0.653	0.746
6 h	1.689	1.703	1.671	1.706
12 h	2.133	2.109	2.130	2.207

Table 4: Model complexity and average CPU training time per horizon. Mean MAE / RMSE are averages over the three horizons.

Model	Params	Avg. train time (s)	Mean MAE	Mean RMSE
LSTM	54,540	37.2	1.159	1.539
GRU	41,100	32.1	1.097	1.488
Seq2Seq	28,289	15.9	1.091	1.485
Attention-LSTM	59,532	36.8	1.164	1.553

10.3 Error growth with horizon

Figure 7 plots MAE and RMSE against the forecast horizon. Errors grow roughly linearly from 1 h to 12 h across all models: MAE grows by a factor of ~ 3.4 (Seq2Seq: $0.47 \rightarrow 1.59$) and RMSE by a factor of ~ 3.2 .

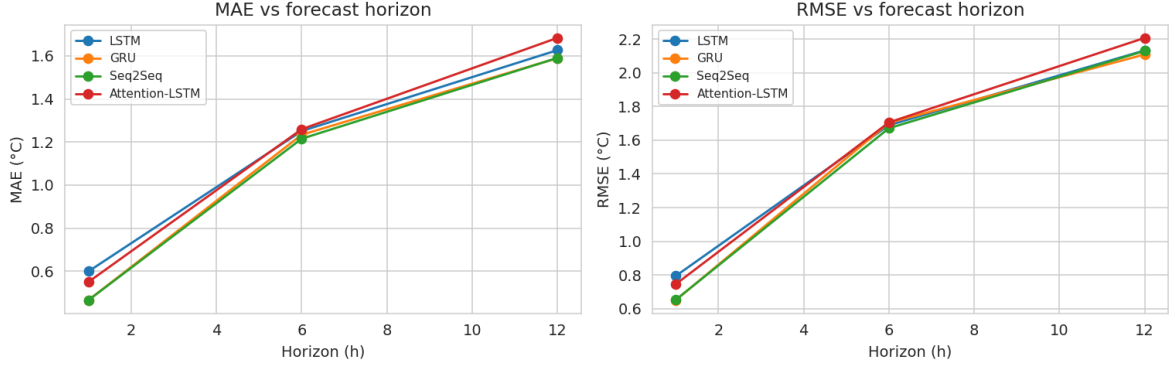


Figure 7: Test MAE (left) and RMSE (right) as a function of the forecast horizon.

10.4 Qualitative forecasts

Figure 8 shows three representative 12-hour test windows. All four models track the underlying temperature trend closely; visible per-model deviations are small compared with the temperature scale.

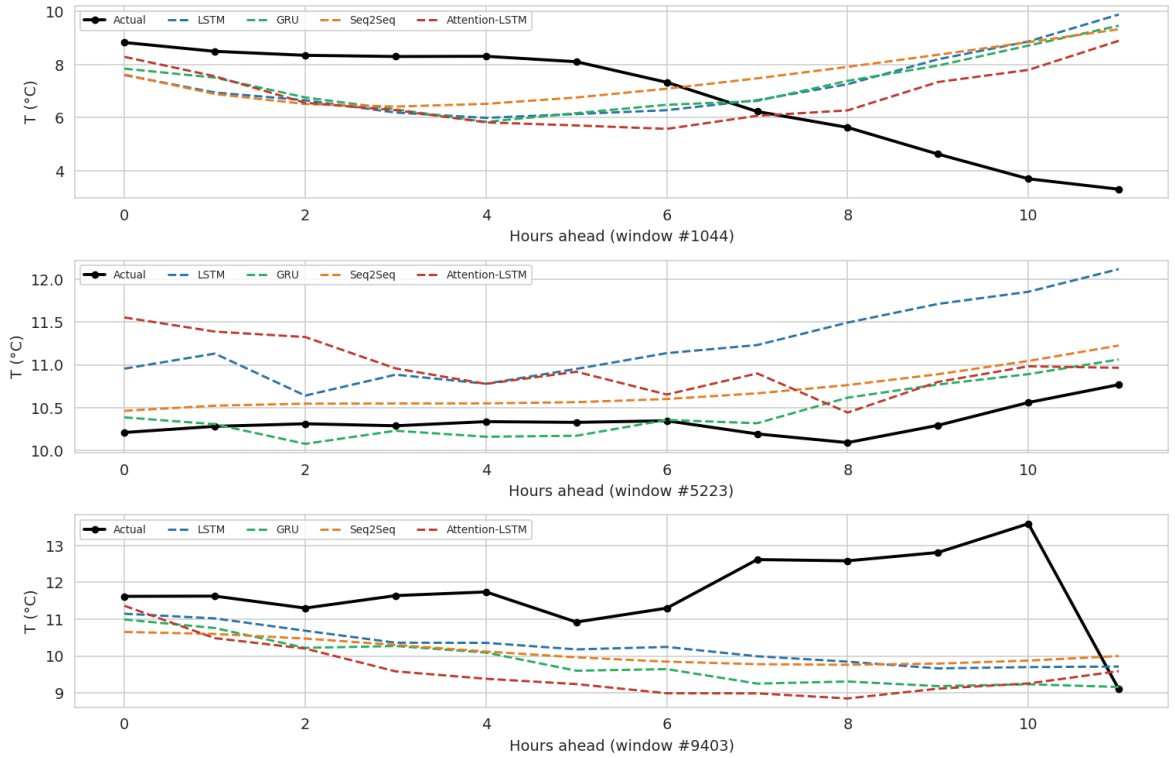


Figure 8: Actual temperature and 12-hour forecasts from the four models on three test windows.

11 Attention Analysis

Figure 9 shows both the mean attention weight per historical hour, averaged over ~ 500 test windows (left), and the attention row of 20 random windows (right). The mean profile is almost flat, sitting in a narrow band around $1/48 \approx 0.021$ (the values displayed on the

colour bar of the right panel span roughly 0.015–0.028). This means that, on average, the trained model gives essentially equal importance to every step in the 48-hour history, and that individual example-level variations are moderate rather than sharply peaked.

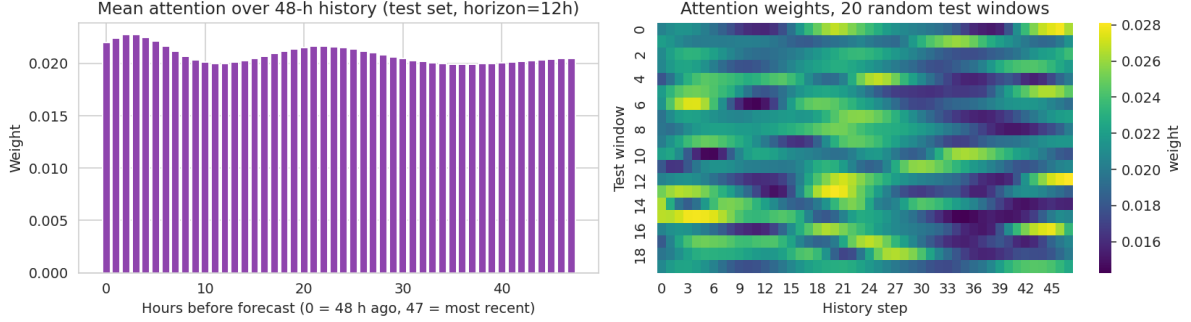


Figure 9: *Left*: mean attention weight over the 48-h history, averaged across the test set at horizon 12 h. *Right*: attention weights for 20 random test windows.

We interpret this cautiously. The attention weights do not prove that different points in history are actually equally informative meteorologically; they only report which hidden states the model chose to emphasise. In our setting the LSTM’s own recurrent dynamics likely already encoded the recency bias needed for this task, so the additional attention head effectively degenerated into a soft-averaging component that did not further help the linear head.

12 Failure Analysis

Figure 10 shows the three test windows with the largest mean $|y - \hat{y}|$ for Attention-LSTM (all in the $> 3\text{--}4^\circ\text{C}$ range). All four models fail together on these windows, and the four prediction curves stay very close to each other while diverging from the truth. These are cases where the 48-hour history evidently does not contain enough information to predict the next 12 hours: sudden weather regime shifts (e.g., front passages that arrive after the history window closes) affect all models equally. We do not claim to have identified the specific meteorological driver of each event — the dataset does not provide auxiliary front / storm labels — but the symptom (all four models fail together, with correlated residuals) is consistent with unpredictable exogenous events rather than a model-specific weakness.

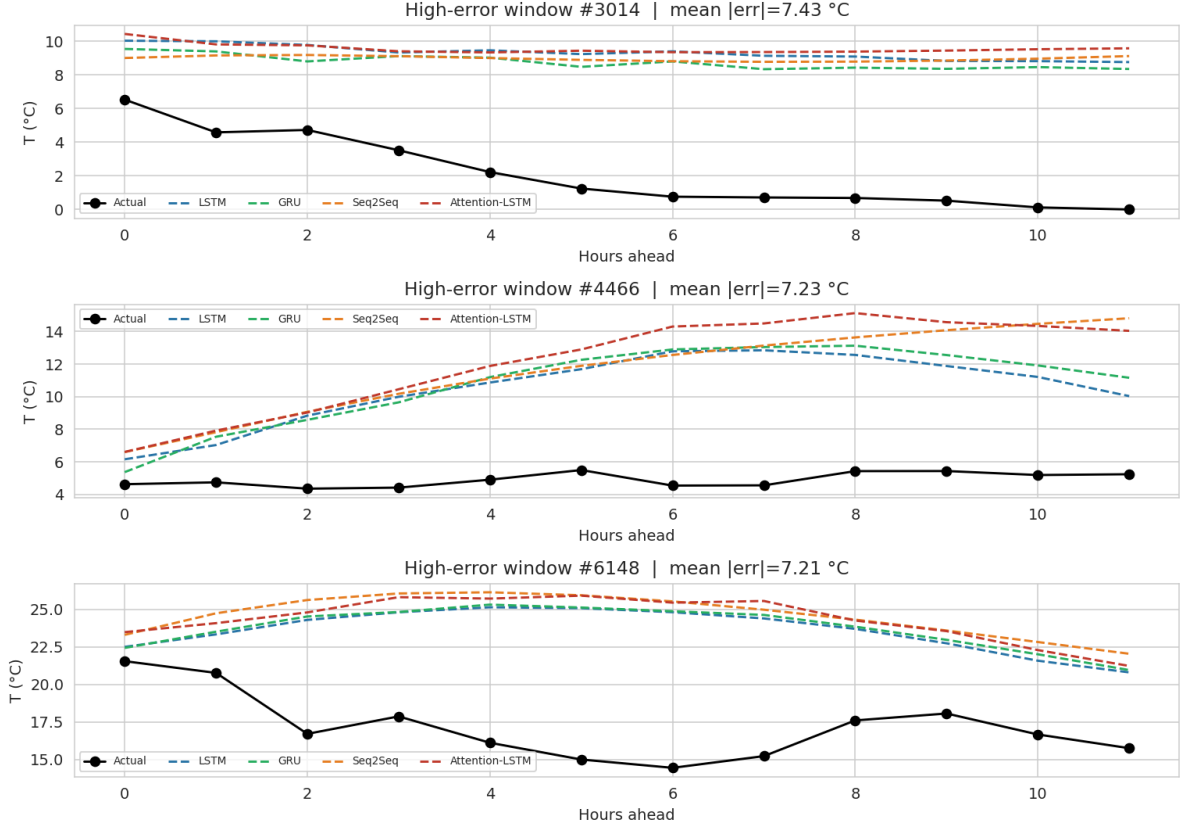


Figure 10: Three test windows with the largest Attention-LSTM error at horizon 12 h. All four models fail together, indicating exogenous events.

13 Model Comparison and Discussion

Which model was best? On mean MAE / RMSE across horizons the minimal Seq2Seq model wins, closely followed by the GRU. The LSTM is consistently third, and the proposed Attention-LSTM comes fourth at every horizon. The differences between GRU and Seq2Seq are within 0.005 °C MAE and are not practically significant; the gap from the LSTM and Attention-LSTM to the best baseline is 0.06–0.13 °C MAE at $h=1$ h.

Did attention help? No. Under this configuration the proposed model neither improved short-horizon nor long-horizon forecasting. The empirical attention distribution (Figure 9) suggests why: it stayed close to uniform, which is roughly equivalent to averaging the LSTM’s hidden states and adds parameters without providing additional useful signal.

RNN vs Transformer. A Transformer [7] also handles ordering (through positional encodings), so the claim is not that Transformers cannot model sequence order — they can. The sequential inductive bias that an RNN provides is the *explicit state transition* $h_{t-1} \rightarrow h_t$: it forces the model to compress history into a running state and encourages sample-efficient learning of short-range dependencies. This is often useful when data is limited or inference must be streaming ($O(d^2)$ per new observation, $O(1)$ memory in the

observation dimension). Transformers, on the other hand, excel with very long sequences, very large datasets and hardware where their parallelism pays off.

For this project (one station, 8 years of hourly data), an RNN is a reasonable default. A Transformer baseline could be an interesting extension but would probably need either a longer input window, more data (multiple stations) or careful regularisation to shine.

When to prefer RNNs. Small datasets, short sequences, strict streaming/latency constraints, embedded deployment, and educational settings. Transformers become clearly preferable at long sequences ($T \gg 10^3$) and when abundant training data is available.

14 Limitations

- **Single station.** All results are for Jena; we did not test cross-station generalisation.
- **Downsampled series.** 10-min $\rightarrow$ 1-h aggregation removes sub-hourly variability. Some very short-term phenomena (gusts, radiative fog dissipation) are averaged out.
- **Compute budget.** Training was capped at 5 epochs on CPU with a stride-3 subsample of training windows. A longer schedule with stride 1 would likely lower every model’s error but is unlikely to change the ranking.
- **No Transformer baseline.** We discuss the RNN vs Transformer contrast but did not train a Transformer.
- **Point forecasts.** We report point predictions only; probabilistic forecasting (prediction intervals) was left as a non-required stretch goal and was not implemented.
- **Limited hyperparameter search.** We picked a single reasonable configuration and did not sweep hidden size, layers or dropout.

15 Conclusions

We built and executed an end-to-end multivariate temperature forecasting pipeline on the Jena Climate dataset, comparing four recurrent architectures. The main findings are:

1. **Best model:** the minimal GRU-based Seq2Seq attained the lowest mean error across horizons (mean MAE 1.09 °C, mean RMSE 1.49 °C), essentially tied with the plain GRU.
2. **GRU vs LSTM:** the GRU used 25 % fewer parameters and matched or beat the LSTM at every horizon — consistent with prior reports that GRUs are competitive with LSTMs on moderate-length sequences.
3. **Seq2Seq:** the simple encoder–decoder was competitive despite having the fewest parameters (28,289), suggesting that on this task most of the value comes from compressing recent history rather than from architectural sophistication.

4. **Attention did not help:** the Attention-LSTM was worse than both simpler baselines at every horizon; its learned attention profile was nearly uniform, showing that under this training budget the mechanism did not identify strongly informative historical steps.
5. **Error growth:** MAE grew roughly linearly with the horizon, from ~ 0.47 °C at $h=1$ h to ~ 1.6 °C at $h=12$ h across models.
6. **Failure mode:** the largest errors occurred on windows where all four models fail together, consistent with unpredictable exogenous weather events rather than model-specific weaknesses.

Future work directions include a Transformer baseline, probabilistic forecasting with e.g. quantile regression, longer training with stride-1 windows, and a proper hyperparameter search.